Insertion Sort in Java

Name: Rodriguez, Miles J.
Date: October 27, 2026
Class Section: BSIT 2H-G1

Activity Objective

- **Cognitive Objective:** Trace and analyze the execution of an ascending Insertion Sort algorithm in Java by evaluating conditional statements, tracking variable updates, and documenting element shifts across iterations.
- **Skill Objective:** Correctly populate tracing tables to demonstrate how elements in an unsorted array are progressively compared, shifted, and inserted into their proper sorted positions.

Instructions

Follow the steps below to complete the Insertion Sort tracing activity based on the provided Java source code:

1. **Analyze the Code:** Review the Insertion Sort program provided. Note how the outer loop picks each element starting from index 1 as the key, and how the inner while loop compares the key with elements in the sorted section from right to left.
2. **Track Iterations:** The algorithm runs through multiple iterations (one for each element in the unsorted section). For each iteration:
    - Identify the current **Key** value and the starting index.
    - Observe the current array state before the inner loop begins.
3. **Fill out the Tracing Table Step-by-Step:**
    - Record the step number and how index changes.
    - Evaluate whether the index condition is true or false.
    - Evaluate whether the element at index is greater than the current key.
    - **Full Condition:** Combine both evaluations using logical AND
    - **Action:** If the condition is **True**, indicate a shift action. If it is **False**, indicate the final placement of the key.
    - **Array State:** Write down the exact layout of the array *immediately after* that step's action is performed.
4. **Verify Final Output:** Ensure that the final array state after all iterations are completed is sorted in ascending order.

Java Sample Code

```
public class InsertionSortAscendingSample {
   public static void main(String[] args) {
      int[] arr = {7, 2, 6, 1, 4};
      insertionSort(arr);
      System.out.print("Final Sorted Array: ");
      for (int num : arr) {
         System.out.print(num + " ");
      }
   }

   static void insertionSort(int[] arr) {
      for (int i = 1; i < arr.length; i++) {
         int key = arr[i];
         int j = i - 1;
         while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
         }
         arr[j + 1] = key;
      }
   }
}
```

Activity Tracing Tables

Initial Array: arr = {7, 2, 6, 1, 4}

Iteration 1 (i = 1)

- **Key** = 2
- **J** = 0
- **Current Array State Before Inner Loop:** {7, 2, 6, 1, 4}

Step	j>=0	arr[j]>key	Full Condition	Action	Array State
1	0 >= 0 → TRUE	7 > 0 → TRUE	TRUE && TRUE → TRUE	SHIFT: arr[1] = arr[0]	{7, 7, 6, 1, 4}
2	j-- → -1			INSERT KEY: arr[0] = 2	{2, 7, 6, 1, 4}

Iteration 2 (i = 2)

- **Key** = 6
- **j** = 1
- **Current Array State Before Inner Loop:** {2, 7, 6, 1, 4}

Step	j>=0	arr[j]>key	Full Condition	Action	Array State
1	1 >= 0 → TRUE	7 > 6 → TRUE	TRUE && TRUE → TRUE	SHIFT: arr[2] = arr[1]	{2, 7, 7, 1, 4}
2	j-- → 1			INSERT KEY: arr[1] = 6	{2, 6, 7, 1, 4}

Iteration 3 (i = 3)

- **Key** = 1
- **J** = 2
- **Current Array State Before Inner Loop:** {2, 6, 7, 1, 4}

Step	j>=0	arr[j]>key	Full Condition	Action	Array State
1	2 >= 0 → TRUE	7 > 1 → TRUE	TRUE && TRUE → TRUE	SHIFT: arr[3] = arr[2]	{2, 6, 7, 7, 4}
2	j-- → 1	6 > 1 → TRUE	TRUE	SHIFT: arr[2] = arr[1]	{2, 6, 6, 7, 4}
3	J-- → 0	2 > 1 → TRUE	TRUE	SHIFT: arr[1] = arr[0]	{2, 2, 6, 7, 4}
4	J-- → -1 J + 1 → 0			INSERT KEY: arr[0] = 1	{1, 2, 6, 7, 4}

Iteration 4 (i = 4)

- **Key** = 4
- **j** = 3
- **Current Array State Before Inner Loop:** {1, 2, 6, 7, 4}

Step	j>=0	arr[j]>key	Full Condition	Action	Array State
1	3 >= 0 → TRUE	7 > 4 → TRUE	TRUE && TRUE → TRUE	SHIFT: arr[4] = arr[3]	{1, 2, 6, 7, 7}
2	j-- → 2	6 > 4 → TRUE	TRUE	SHIFT: arr[3] = arr[2]	{1, 2, 6, 6, 7}
3	j-- → 1	2 > 4 → FALSE	FALSE	INSERT KEY: arr[2] = 4	{1, 2, 4, 6, 7}

Final Sorted Array: {1, 2, 4, 6, 7}

Grading Rubric

Criteria	Excellent (4 Points)	Proficient (3 Points)	Developing (2 Points)	Beginning (1 Point)
Accuracy of Table Logic (j≥0 & arr[j]>key)	All conditions, boolean evaluations, and logical combinations are 100% accurate across every iteration.	Minor errors in evaluating boolean conditions, but core logic is mostly correct.	Frequent errors in evaluating array elements against the key or managing boundary conditions (j≥0).	Conditions are largely incorrect, showing a fundamental misunderstanding of the inner loop logic.
Tracing Actions & Array States	All shifts, key insertions, and resulting array states are documented flawlessly step-by-step.	Array states are mostly correct with 1 or 2 minor transcription or indexing mistakes during shifts.	Several errors in tracking element shifts and updating the array state, resulting in a flawed final array.	Array states are blank, unorganized, or completely disconnected from the algorithm's actions.
Completeness	All iterations (i=1 through the end of the array) are fully completed with no missing fields or steps.	Nearly all iterations are complete, with at most one minor step omitted.	Multiple iterations or required columns are left blank or incomplete.	The activity is largely incomplete or missing substantial sections.
Adherence to Lesson Format	Follows the exact structural layout, notation, and table headings demonstrated in the lesson PDF.	Follows the format closely with minor formatting deviations.	Strays significantly from the expected lesson format, making verification difficult.	Does not follow the lesson format at all.